

Arquitecturas Generativas

Guia de Estudio

VAE - GAN - Modelos de Difusion - Flujos Normalizadores

Fundamentos matematicos, funciones de costo y aplicaciones

Bienvenida a la guía

Este documento está diseñado para estudiantes de ingeniería industrial que desean comprender las **cuatro arquitecturas principales de inteligencia artificial generativa** que están redefiniendo la automatización, el diseño y el mantenimiento industrial. El enfoque es pedagógico: partimos de los fundamentos probabilísticos, desglosamos cada componente de cada arquitectura, derivamos sus funciones de costo y aterrizamos los conceptos con ejemplos concretos de la ingeniería.

Objetivos de aprendizaje al finalizar esta guía:

- » Identificar los componentes matemáticos de **VAE, GAN, Modelos de Difusión y Flujos Normalizadores**.
- » Interpretar y derivar las **funciones de costo** (ELBO, juego min-max, denoising, log-verosimilitud exacta).
- » Relacionar cada arquitectura con problemas reales de **control de calidad, mantenimiento predictivo, diseño generativo y análisis de riesgos**.
- » Elegir la arquitectura adecuada según la naturaleza del problema industrial planteado.

Referencia principal: Bishop, C. M. y Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer. Capítulos 17 (GAN), 18 (Normalizing Flows), 19 (Autoencoders y VAE) y 20 (Diffusion Models).

Contenido

0	Conceptos base de probabilidad	04
1	Variational Autoencoders (VAE)	05
2	Generative Adversarial Networks (GAN)	08
3	Modelos de Difusion (DDPM)	10
4	Flujos Normalizadores (Normalizing Flows)	12
5	Cuadro comparativo y guía de elección	14

Como usar esta guía: cada capítulo sigue la misma estructura pedagógica — primero **intuición**, luego **componentes matemáticos** desglosados, después **la función de costo derivada paso a paso**, y al final **un ejemplo industrial** que conecta la teoría con la práctica.

00

Conceptos base de probabilidad

Antes de abordar las redes neuronales generativas, debemos entender el **lenguaje en el que hablan los datos**. Los modelos que veremos no aprenden a copiar; aprenden a **reproducir la distribución de probabilidad subyacente** que generó los datos. Tres conceptos son imprescindibles.

Distribución de probabilidad $p(x)$

Es una función que asigna un valor a cada posible resultado del experimento indicando que tan probable es observarlo. En ingeniería, x representa los datos (mediciones de sensores, imágenes de piezas, series temporales). El **objetivo de todo modelo generativo es aprender la forma exacta de $p(x)$** para luego muestrear nuevos ejemplos de esa misma distribución.

Verosimilitud (Likelihood)

Mide que tan probable es que nuestro modelo, con sus parámetros w actuales, haya producido los datos que efectivamente observamos. **Optimizar un modelo generativo equivale (en la mayoría de los casos) a maximizar la verosimilitud** de los datos. Cuando el cálculo directo es imposible, recurrimos a límites inferiores (como ELBO) o a juegos adversarios (como GAN).

Espacio latente z

Un plano matemático comprimido y oculto. En lugar de procesar una imagen pixel a pixel (alta dimensionalidad), la reducimos a unas pocas variables que describen su **esencia** (grosor, curvatura, textura, orientación). En los modelos generativos, se asume una distribución simple sobre z (típicamente Gaussiana estándar) y una red neuronal no lineal transforma esa simplicidad en la riqueza de los datos reales.

El marco común de las cuatro arquitecturas

Las cuatro arquitecturas que estudiaremos comparten una idea central: **transformar una distribución simple $p(z)$ en una distribución compleja $p(x)$ mediante una red neuronal**. La diferencia entre ellas radica en **como se entrena la red** y **como se maneja la intratabilidad de la verosimilitud**.

Arquitectura	Estrategia de entrenamiento	Verosimilitud
VAE	Límite inferior variacional (ELBO)	Aproximada
GAN	Juego adversario (min-max)	Implícita
Difusión	Denoising iterativo	ELBO jerárquico
Flujos	Cambio de variables con Jacobiano	Exacta

01 Variational Autoencoders (VAE)

Intuición

Un VAE comprime los datos a un **espacio probabilístico** y luego intenta reconstruirlos. A diferencia de un autoencoder clásico, el VAE no aprende un punto único en el espacio latente para cada entrada, sino **una distribución de probabilidad** sobre z . Esto obliga al modelo a organizar el espacio latente de forma continua y semántica: puntos cercanos en z generan salidas similares en x .

Componentes

Encoder $q_{\phi}(z | x)$

Red neuronal que toma el dato de entrada x (por ejemplo, una imagen de una pieza) y no entrega un punto fijo, sino los **parámetros de una distribución normal**: una media $\mu(x, \phi)$ y una varianza $\Sigma(x, \phi)$. Se puede pensar como un **compresor probabilístico** que dice, para cada entrada, en qué región del espacio latente podría habitar.

Decoder $p_{\theta}(x | z)$

Red neuronal que toma una muestra z del espacio latente y **reconstruye el dato original**. Típicamente produce la media de una distribución Gaussiana sobre x (aunque puede parametrizar otras distribuciones). Es el componente que finalmente se usa para **generar muestras nuevas**: se descarta el encoder y se muestrea $z \sim p(z)$ para obtener x mediante el decoder.

Muestra latente z^*

El punto específico que tomamos aleatoriamente de la distribución $q_{\phi}(z|x)$ creada por el encoder para pasárselo al decoder. Este muestreo es lo que hace al VAE **probabilístico**: cada pase por la red genera una reconstrucción ligeramente distinta.

El truco de reparametrización

Problema: si muestreamos z directamente de $q_{\phi}(z|x)$, **no podemos propagar gradientes** a través de la operación de muestreo (el azar no es diferenciable). Sin gradientes, el encoder no puede aprender.

Solución: separamos la aleatoriedad de los pesos. En lugar de muestrear z , muestreamos ruido $\epsilon \sim N(0, I)$ y construimos z aplicando una transformación determinista sobre ϵ y los parámetros predichos por la red:

$$z^* = \mu(x, \phi) + \Sigma^{1/2}(x, \phi) \varepsilon, \text{ con } \varepsilon \sim N(0, I)$$

La aleatoriedad se inyecta por un lado; los parametros μ y Σ siguen siendo diferenciables.

Construccion de la funcion de costo: ELBO

Queremos maximizar $\log p(x)$, pero calcular esto requiere integrar sobre todas las posibles z , lo cual es **intratable**. En su lugar, maximizamos un limite inferior, el **Evidence Lower Bound (ELBO)**, que siempre satisface $L(w, \phi) \leq \log p(x)$.

$$L = E_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)] - D_{KL}(q_{\phi}(z|x) \parallel p(z))$$

Evidence Lower Bound (ELBO): dos terminos con interpretaciones muy claras.

Termino 1: reconstruccion

$E_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)]$ mide **que tan bien el decoder reconstruye el dato original** cuando se le pasa una muestra z del encoder. Si $p(x|z)$ es Gaussiana con varianza fija, este termino se convierte (salvo constantes) en un **error cuadratico medio (MSE)** entre la entrada y la reconstruccion.

Termino 2: divergencia KL (regularizador)

$D_{KL}(q_{\phi}(z|x) \parallel p(z))$ es una **penalizacion** que obliga a que la distribucion producida por el encoder se parezca a la distribucion prior (tipicamente $N(0, I)$). Sin este termino, el VAE colapsaria a un autoencoder determinista y el espacio latente perderia su estructura. Con este termino, el espacio latente queda **ordenado y continuo**, lo que habilita la generacion de nuevas muestras coherentes.

Cuando tanto $q_{\phi}(z|x)$ como $p(z)$ son Gaussianas, la divergencia KL tiene forma cerrada y se puede calcular exactamente sin muestreo:

$$D_{KL} = \frac{1}{2} \sum_j [1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2]$$

Forma cerrada cuando el prior es $N(0, I)$ y el posterior tambien es Gaussiano diagonal.

Metricas de evaluacion

- » **Error de reconstruccion (MSE / error cuadratico medio):** mide la fidelidad del decoder.
- » **Divergencia KL:** mide que tan «ordenado» quedo el espacio latente.
- » **Log-verosimilitud estimada:** se usa el propio ELBO como aproximacion de $\log p(x)$.

EJEMPLO APLICADO » Control de calidad por reconstrucción

Contexto: una línea de montaje produce piezas de acero. Queremos detectar automáticamente piezas defectuosas (fisuras, deformaciones) sin tener muchos ejemplos de defectos.

Solución con VAE: se entrena el VAE **únicamente con piezas perfectas**. El modelo aprende un espacio latente que codifica solo la «geometría de la perfección».

Operación: cuando en la línea ingresa una pieza con fisura, al pasarla por el VAE el **error de reconstrucción se dispara** porque esa geometría no habita en el espacio latente aprendido. El sistema marca la pieza como anómala.

Umbral de decisión: se fija empíricamente a partir del percentil 99 del error en el conjunto de piezas sanas. Todo error por encima de ese umbral dispara una alarma.

02

Generative Adversarial Networks (GAN)

Intuición

Las GAN no modelan probabilidades de forma explícita: aprenden a generar datos creando una **competencia entre dos redes neuronales**. Una red **generador G** produce datos sintéticos intentando enganar; otra red **discriminador D** intenta detectar si el dato es real o falso. La tensión entre ambas empuja a G a producir muestras cada vez más realistas.

Componentes

Generador $g(z, w)$

Red neuronal profunda que toma un vector de ruido $z \sim N(0, I)$ y lo transforma en un dato sintético x . No ve datos reales directamente: solo recibe retroalimentación a través del discriminador. Su objetivo es **maximizar el error del discriminador**.

Discriminador $d(x, \phi)$

Red neuronal clasificadora binaria con activación sigmoide final. Su salida interpreta $P(t=1 | x)$, es decir, la probabilidad de que x sea real. Se entrena con **entropía cruzada binaria** sobre pares etiquetados (real / sintético). Su objetivo es **minimizar ese error**.

El juego min-max (teoría de juegos)

Lo que uno gana, el otro lo pierde: estamos ante un **juego de suma cero**. Formalmente, G y D optimizan de manera opuesta una misma función $V(D, G)$. G busca minimizar; D busca maximizar.

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$

Función objetivo del GAN clásico (Goodfellow et al., 2014).

Desglose de la función de costo

Término 1: $E_{x \sim p_{\text{data}}} [\log D(x)]$. D debe asignar probabilidad alta a datos reales. D lo **maximiza**.

Término 2: $E_{z \sim p_z} [\log(1 - D(G(z)))]$. D debe asignar probabilidad baja a datos sintéticos; G busca exactamente lo contrario. Por eso G **minimiza** la función completa (o equivalentemente maximiza $\log D(G(z))$ en la formulación práctica, que tiene mejores gradientes al inicio).

El equilibrio de Nash

Es el **punto de estancamiento perfecto**: G crea datos tan indistinguibles de la realidad que D se ve forzado a responder con probabilidad 0.5 (equivalente a **lanzar una moneda al aire**). En ese punto, $p_G(x) = p_{data}(x)$ y ninguno de los dos puede mejorar unilateralmente cambiando su estrategia.

Desafíos de entrenamiento

- » **Mode collapse**: G descubre un puñado de salidas que siempre engañan a D y deja de explorar la diversidad del dataset.
- » **Gradientes desvanecidos**: cuando D es «demasiado bueno», el término $\log(1 - D(G(z)))$ tiene gradiente casi nulo.
- » **Sin métrica de progreso monotono**: la función de costo oscila por diseño; se necesitan métricas externas como FID.
- » **Soluciones prácticas**: Wasserstein GAN (con gradient penalty), Least-Squares GAN, instance noise, formulación alternativa $\max \log D(G(z))$.

Métricas de evaluación

- » **Frechet Inception Distance (FID)**: compara la distribución de características (extraídas con una red pre-entrenada) entre datos reales y generados. Menor es mejor.
- » **Inspección visual y métricas de dominio**: en aplicaciones industriales, la utilidad del dato sintético se valida reentrenando modelos downstream y comparando desempeño.

EJEMPLO APLICADO » Mantenimiento predictivo por aumentación de datos

Contexto: se tienen sensores de vibración en turbinas eólicas, pero los **fallos críticos son raros**. La base de datos tiene 99% de muestras de operación normal y menos del 1% de fallo: imposible entrenar un clasificador robusto con ese desbalance.

Solución con GAN condicional: se entrena una GAN que toma como entrada la etiqueta «fallo» y aprende a generar **series temporales sintéticas de vibración durante fallos** que son estadísticamente indistinguibles de las pocas muestras reales.

Uso downstream: con un dataset aumentado (real + sintético), se entrena un clasificador de alarmas predictivas que ahora tiene suficiente exposición al patrón de fallo para detectarlo con días de anticipación.

Precaución: antes de usar datos sintéticos en producción, se valida que el clasificador entrenado sobre ellos conserve el desempeño al probarse en el pequeño conjunto de fallos reales.

03

Modelos de Difusion (DDPM)

Intuicion

Inspirados en la **termodinamica molecular**, los modelos de difusion destruyen los datos gradualmente con ruido Gaussiano y luego aprenden a **revertir ese proceso**. Una red neural no aprende a «dibujar desde cero»; aprende a **predecir cuanto ruido se inyecta en cada paso** para poder restarlo y limpiar la señal progresivamente.

Proceso forward: cadena de Markov ruidosa

Una **cadena de Markov** es una secuencia de eventos donde cada paso depende *solo* del paso inmediatamente anterior. En nuestro caso, tomamos un dato x y le inyectamos ruido Gaussiano de manera iterativa, T veces, hasta que la ultima variable z_T sea **estatica pura** indistinguible de $N(0, I)$.

$$q(z_t | z_{t-1}) = N(z_t; \sqrt{(1-\beta_t)} z_{t-1}, \beta_t I)$$

En cada paso se aplica un poco de ruido Gaussiano con varianza β_t .

Una propiedad muy util: se puede **saltar a cualquier paso t directamente** sin simular toda la cadena, gracias a la propiedad de composicion de Gaussianas. Si definimos $\alpha_t = \prod_{\tau=1..t} (1-\beta_\tau)$, entonces:

$$q(z_t | x) = N(z_t; \sqrt{\alpha_t} x, (1-\alpha_t) I)$$

Kernel de difusion: acceso directo a cualquier nivel de ruido, esencial para entrenamiento eficiente.

Proceso reverse: la red aprende a denoise

Una red neural (tipicamente una **U-Net**) toma como entrada z_t y el indice de paso t , y **predice el ruido total ϵ que se anadio a x para obtener z_t** . La red no aprende a dibujar: aprende a estimar ruido.

Funcion de costo: prediccion de ruido

La funcion de costo, derivada simplificando el ELBO de un autoencoder variacional jerarquico, resulta elegantemente simple:

$$L(w) = E_{t, x, \epsilon} [\| \epsilon - \epsilon_w(\sqrt{\alpha_t} x + \sqrt{(1-\alpha_t)} \epsilon, t) \|^2]$$

Error cuadratico medio entre el ruido real y el ruido predicho por la red.

Desglose de la funcion de costo

- » **t aleatorio** en $\{1, \dots, T\}$: la red se expone a todos los niveles de ruido durante el entrenamiento.
- » **x** muestreado del conjunto de entrenamiento: el dato limpio original.
- » $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: ruido Gaussiano muestreado frescamente.
- » $\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x} + \sqrt{(1-\alpha_t)} \varepsilon$: la version corrupta construida con el kernel de difusion.
- » $\varepsilon_w(\mathbf{z}_t, t)$: prediccion de la red, del mismo tamaño que x.
- » El objetivo es que la red «adivine» el ruido; si lo logra, podemos restarlo y recuperar x.

Como se generan nuevas muestras

(1) Se muestrea $\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. (2) Para $t = T, T-1, \dots, 1$ se aplica el paso de denoising usando la red entrenada: $\mathbf{z}_{t-1} = \mu(\mathbf{z}_t, w, t) + \sqrt{\beta_t} \varepsilon'$ con $\varepsilon' \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (salvo en el ultimo paso donde no se añade ruido). (3) La salida final es una nueva muestra en el espacio de datos.

Metricas de evaluacion

- » **Error de prediccion de ruido (MSE)**: metrica primaria durante el entrenamiento.
- » **Frechet Inception Distance (FID)**: calidad de las muestras generadas.
- » **Inception Score (IS)**: diversidad y definicion de clase (cuando hay etiquetas).

EJEMPLO APLICADO » Diseño generativo topologico

Contexto: diseño topologico de piezas aeronauticas. Se debe crear la estructura metalica **mas ligera** posible que soporte ciertas cargas (tension maxima, torsion, impacto) en geometrias restringidas.

Solucion con modelo de difusion condicional: el modelo se entrena con miles de piezas validadas por simulacion de elementos finitos. Al muestrear, se le provee como **condicion** el vector de restricciones mecanicas (cargas, puntos de apoyo, volumen disponible).

Operacion intuitiva: se parte de una forma inicial de ruido puro ($\mathbf{z}_T$) y la red, en el proceso reverse, va «**restando material**» paso a paso hasta dejar unicamente la estructura optima. Es literalmente escultura estadistica.

Validacion: cada pieza generada se verifica via FEM antes de fabricacion. La difusion no reemplaza al ingeniero: le propone candidatos optimizados que el analiza.

04

Flujos Normalizadores (Normalizing Flows)

Intuición

A diferencia de VAE, GAN y difusión, los **flujos normalizadores** ofrecen **exactitud matemática impecable**: permiten calcular la log-verosimilitud $\log p(x)$ **directamente, sin aproximaciones**. El precio es una restricción arquitectónica: todas las transformaciones deben ser **biyectivas e invertibles**.

Transformación biyectiva

Una función $f: z \rightarrow x$ es **biyectiva** si cada punto en z tiene un único punto correspondiente en x y viceversa (como estirar una banda elástica sin romperla ni doblarla sobre sí misma). Formalmente: existe g tal que $g(f(z)) = z$ para todo z , y $f(g(x)) = x$ para todo x .

Cambio de variables: la fórmula clave

Si $z \sim p_z(z)$ (distribución simple, p.ej. Gaussiana) y $x = f(z)$ es biyectiva, entonces la densidad de x se obtiene por el **teorema del cambio de variables** usando el determinante del Jacobiano:

$$p_x(x) = p_z(g(x)) \cdot |\det J_g(x)|$$

El Jacobiano J_g tiene como entradas $\partial g / \partial x_i$. Es una matriz $D \times D$.

Aplicando logaritmo (más estable numéricamente) y sumando sobre N datos independientes, la log-verosimilitud se vuelve la función de costo a **maximizar directamente**:

$$\log p(D | w) = \sum_{n=1..N} [\log p_z(g(x_n, w)) + \log |\det J(x_n)|]$$

Función objetivo de los flujos normalizadores: log-verosimilitud exacta (no es un límite inferior).

Desglose de la función de costo

- » **$\log p_z(g(x))$** : valora que tan «típico» es el punto latente asociado a x bajo la distribución base (p.ej. $N(0, I)$, donde $\log p_z(z) = -||z||^2/2 + \text{const}$).
- » **$\log |\det J(x)|$** : es el **factor de compensación de volumen**. Si la transformación comprime el espacio localmente, concentra probabilidad; si lo estira, la dispersa.
- » **Por que no hay ELBO ni juego adversario**: la invertibilidad elimina la necesidad de aproximar; la densidad se calcula exactamente.

Restricciones arquitectónicas

Para que la red sea invertible y el Jacobiano calculable eficientemente, se imponen formas especiales como **coupling flows** (Real NVP: se parte el vector en dos, una mitad se copia y la otra se transforma linealmente condicionada por la primera) o **flujos autorregresivos** (cada dimension se transforma condicionada por las previas). La dimensionalidad del espacio latente **debe ser igual** a la del espacio de datos.

Metricas de evaluacion

» **Log-Verosimilitud Negativa (NLL)**: la metrica natural. Cuanto menor, mejor representa el modelo a la distribucion real.

» **Bits per dimension (BPD)**: NLL normalizada por dimension, comun en benchmarks de imagenes.

EJEMPLO APLICADO » Analisis de riesgos en planta petroquimica

Contexto: en una planta petroquimica, las variables de **presion, temperatura y flujo de valvulas** tienen interacciones no lineales muy complejas. Los modelos estadisticos clasicos (Gaussianas multivariadas, copulas) fallan en capturar **eventos de cola pesada** y cascadas de fallos.

Solucion con flujo normalizador: se entrena el modelo sobre datos historicos de operacion normal y de incidentes. A diferencia de VAE o GAN, aqui **obtenemos la densidad conjunta exacta** $p(\text{presion, temperatura, flujo, ...})$.

Operacion: en tiempo real, para cada vector de mediciones x , calculamos $p(x)$. Si el valor cae por debajo de un umbral (evento raro), se emite una **alerta de configuracion anomala** con **precision decimal** sobre la rareza del evento.

Ventaja decisiva: como tenemos la densidad exacta, podemos integrar sobre regiones para calcular probabilidades de eventos compuestos (p.ej. probabilidad conjunta de cascada de fallos), algo que ni VAE ni GAN permiten hacer rigurosamente.

05

Cuadro comparativo y guía de elección

Tabla síntesis

Dimension	VAE	GAN	Difusion	Flujos
Funcion de costo	ELBO (MSE + KL)	Juego min-max (entropia cruzada)	MSE de ruido predicho	Log-verosimilitud exacta
Verosimilitud	Aproximada	Implicita	Aproximada (ELBO jerarquico)	Exacta
Calidad de muestras	Media (borrosas)	Alta (pero colapsa)	Muy alta (estado del arte)	Media-alta
Velocidad de muestreo	Rapida (1 pase)	Rapida (1 pase)	Lenta (T pases)	Rapida-media
Espacio latente	Semantico continuo	Semantico pero irregular	Jerarquico temporal	Determinista invertible
Aplicacion tipica	Deteccion de anomalias	Aumentacion de datos raros	Diseño generativo condicional	Análisis de riesgos / densidad

Guía rápida de elección

No hay una arquitectura universalmente superior. La elección depende del problema industrial específico. Estas heurísticas ayudan a orientarse:

¿Necesitas detectar anomalías a partir de datos «normales»?

Usa **VAE**. Entrenas solo con operación normal, y el error de reconstrucción actúa como detector de anomalías. Alternativa: flujos normalizadores, si quieres probabilidades exactas.

¿Necesitas aumentar un dataset con muestras raras (fallos, defectos)?

Usa **GAN condicional**. Genera muestras sintéticas conjuntamente con su etiqueta para balancear clases. Valida siempre con un holdout de datos reales.

¿Necesitas generar diseños que cumplan restricciones físicas?

Usa **modelos de difusión condicionales**. Son estado del arte en calidad y permiten incorporar condiciones complejas (cargas, geometrias, textos descriptivos).

¿Necesitas calcular probabilidades exactas de eventos compuestos?

Usa **flujos normalizadores**. Son los únicos que te dan $\log p(x)$ sin aproximaciones, imprescindible para análisis de riesgos cuantitativos y eventos de cola pesada.

Reflexión final: estas cuatro familias no compiten por un trono. Comparten el mismo marco conceptual (transformar $p(z)$ en $p(x)$ mediante redes neuronales profundas) y difieren en el balance **expresividad / tratabilidad / velocidad**. Dominar las cuatro te da un abanico de herramientas para cualquier problema generativo en ingeniería: desde la aumentación de datos y el control de calidad hasta el diseño topológico y el análisis cuantitativo de riesgos.

Para profundizar

- » Bishop, C. M. y Bishop, H. (2024). **Deep Learning: Foundations and Concepts**. Springer. Cap. 17-20.
- » Goodfellow, I. et al. (2014). **Generative Adversarial Networks**. NeurIPS.
- » Kingma, D. P. y Welling, M. (2013). **Auto-Encoding Variational Bayes**. ICLR.
- » Ho, J., Jain, A. y Abbeel, P. (2020). **Denoising Diffusion Probabilistic Models**. NeurIPS.
- » Dinh, L., Sohl-Dickstein, J. y Bengio, S. (2016). **Density estimation using Real NVP**. ICLR.
- » Song, Y. y Ermon, S. (2019). **Generative Modeling by Estimating Gradients of the Data Distribution**. NeurIPS.